

SOFTWARE ENGINEERING

مقدمة

Abdulfattah Alfarra

OUTLINE

□ البرمجيات

□ البرامج المهنية مقابل البرامج الطلابية

□ درجة الصعوبة لمشروع برمجي □ هندسة البرمجيات □ فشل البرمجيات

□ بناء أو شراء □ قوانين هندسة البرمجيات

Software

البرمجيات - حسب تعريف معهد مهندسي الكهرباء والإلكترونيات

البرنامج هو:

1) برامج الكمبيوتر (الرمز)

2) الإجراءات (3) الوثائق (4) البيانات التي تؤثر على تشغيل نظام

الكمبيوتر.

Software

□ هناك حاجة إلى جميع المكونات الأربعة في عملية تطوير البرمجيات وخدمات الصيانة للسنوات القادمة للأسباب التالية:

(1) هناك حاجة إلى **برامج الكمبيوتر** (الرمز) لأنها، بطبيعة الحال، تعمل على تنشيط الكمبيوتر لأداء التطبيقات المطلوبة.

Software

(2) هناك حاجة إلى إجراءات لتحديد الترتيب والجدول الزمني الذي يتم به تنفيذ البرامج، والطريقة المستخدمة، والشخص المسؤول عن تنفيذ الأنشطة اللازمة لتطبيق البرنامج.

Software

٣) هناك حاجة إلى أنواع مختلفة من الوثائق للمطورين والمستخدمين وفريق الصيانة. تتيح وثائق التطوير (تقرير المتطلبات، تقارير التصميم، أوصاف البرامج، إلخ) تعاونًا وتنسيقًا فعالين بين أعضاء فريق التطوير، ومراجعات وفحوصات فعّالة لمنتجات التصميم والبرمجة. □ وثائق المستخدم (دليل المستخدم)

،
إلخ) يوفر وصفًا للتطبيقات المتاحة والطريقة المناسبة لها

يستخدم.

Software

(4) **البيانات** بما في ذلك المعلومات والرموز وقوائم الأسماء التي تكيف البرنامج مع احتياجات المستخدم المحدد ضرورية لتشغيل البرنامج.

□ نوع آخر من البيانات الأساسية هو بيانات الاختبار القياسية،

□ كما أن هناك أنواعًا من الأنظمة يجب أن تتضمن البيانات مثل الأنظمة المستندة إلى الويب، وأنظمة الخبراء، وأنظمة استخراج البيانات.

Efforts of software Engineering Activates

● Project Management	8.08%
● Requirements	14.43%
● Design	11.36%
● Coding	13.50%
● SQA	30.64%
● SCM	13.02%
● Integration	6.54%
● Misc.	~3%

Types of Software

• قد تكون منتجات البرمجيات **• عامة** - تم تطويرها ليتم بيعها لمجموعة من العملاء المختلفين

على سبيل المثال، برامج الكمبيوتر مثل Excel أو Word. • **مخصص** - تم تطويره لعميل واحد وفقًا لمواصفاته.

قد يكون العميل خارجيًا أو داخليًا

Professional Vs. Student Software

يمكن للطالب بناء نظام برمجي لمدة 2 إلى 3 شهور

قد يستغرق بناء النظام نفسه عشرة أشهر من قبل مهندس برمجيات، لماذا؟
الإجابة ، بالطبع، هي أن هناك شيئين مختلفين

يتم بناؤها في السيناريوهين.

في الحالة الأولى، يجري بناء نظام طلابي مُصمم أساسًا لأغراض العرض التوضيحي، ومن غير المتوقع استخدامه لاحقًا. ولأنه غير مُخصص للاستخدام، فلا يعتمد أي شيء ذي أهمية على البرنامج، ولا يُشكل وجود الأخطاء البرمجية أو ضعف الجودة مصدر قلق كبير. وكذلك الحال بالنسبة لمسائل الجودة الأخرى، مثل سهولة الاستخدام والصيانة والنقل، وما إلى ذلك.

Professional Vs. Student Software

من ناحية أخرى، يتم بناء نظام برمجي **قوي صناعيًا** لحل بعض مشكلات العميل ويستخدمه منظمة العميل لتشغيل جزء من العمل، ويمكن أن يؤدي خلل في مثل هذا النظام إلى تأثير كبير من حيث الخسارة المالية أو التجارية، أو الإزعاج للمستخدمين، أو خسارة الممتلكات والحياة.

وبالتالي، يجب أن يكون نظام البرمجيات عالي الجودة فيما يتعلق بخصائص مثل الموثوقية، وسهولة الاستخدام، والقدرة على النقل، وما إلى ذلك.

إن الحاجة إلى الجودة العالية وإرضاء المستخدمين النهائيين لها تأثير كبير على طريقة تطوير البرمجيات وتكلفتها. تشير القاعدة العامة إلى أن تكلفة البرمجيات عالية الجودة قد تبلغ حوالي عشرة أضعاف تكلفة برمجيات الطلاب.

Degree of Difficulty for software project

عالي	معتدل	قليل	صفات
كبير	واسطة	عدد الوظائف التي يؤديها الصغير أو (LOC)	
نظرية جديدة	مشابه ل النظام الحالي	معايير الوظائف	
كثير	عديد	عدد من 1 الوصول المتزامن	
نعم	بعض	لا	تعدد المهام
عاليا	بعض	حزمة	الوصول التفاعلي مقابل الوصول الدفعي
في الوقت الحالي	معتدل	غير متصل	متطلبات وقت الاستجابة

Degree of Difficulty for software project

عالي	معتدل	قليل	صفات
كثير	جهازي كمبيوتر	الحاجة للتوزيع لا شيء	
كبير	واسطة	كمية البيانات المخزنة صغيرة	
معقد علاقة	معتدل	علاقة بيانات بسيطة	بنية البيانات
عالي	معتدل	قليل	دقة البيانات
هل يمكن التعامل لفترة قصيرة؟ هل يمكن التسامح مع عدم عدة ساعات وجود وقت للتوقف؟			التسامح مع التوقف
عالي	معتدل	لا أحد	احتياجات الأمن

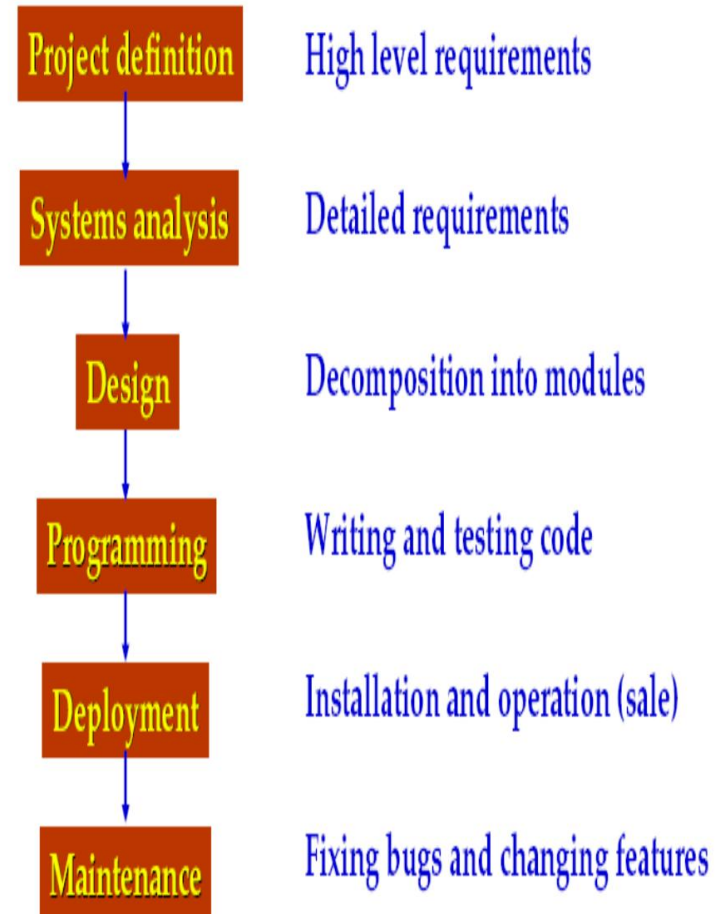
Degree of Difficulty for software project

عالي	معتدل	قليل	صفات
كثيراً	بعضها محدد جيداً	التفاعل مع الآخرين لا شيء الأنظمة	
مرتبطة بموضوع محدد	بعض المستقلين	الاعتماد على الأجهزة	
متكرر يتغير	بعض التغييرات	متطلبات العمل المتغيرة الثابتة تحديد	
مألوف مع السذاجة أنظمة أخرى مماثلة		مستوى تطور المستخدم مألوف	
الخبرة مع عدم وجود خبرة الأدوات، وليس الأنظمة		وقد طورت أنظمة مماثلة	المطور التطور

Software Engineering

ما هي هندسة البرمجيات ؟ هندسة
البرمجيات هي تخصص هندسي يهتم
بجميع جوانب إنتاج البرمجيات (ليس فقط
البرمجة والترميز).

أو



Software Engineering

لماذا هندسة البرمجيات؟ في هندسة البرمجيات ، لا نتعامل مع برامج يُصممها الناس لتوضيح شيء ما أو للتسلية (والتي نشير إليها بأنظمة الطلاب). بل إن مجال المشكلة هو البرمجيات التي تحل بعض مشاكل المستخدمين، حيث قد تعتمد الأنظمة أو الشركات الأكبر حجمًا عليها، وحيث قد تؤدي مشاكلها إلى خسائر كبيرة، مباشرة أو غير مباشرة. نشير إلى هذه البرمجيات ببرمجيات القوة الصناعية.

Software Engineering

هندسة البرمجيات هي تخصص يهدف إلى إنتاج برمجيات عالية الجودة، وتسليمها في الوقت المحدد، وفي حدود الميزانية، وتلبية احتياجات المستخدمين

Software Engineering

□ ظهرت هندسة البرمجيات في الأصل ك
الاستجابة لأزمة تطوير البرمجيات
الستينيات. ثم تطورت لاحقًا لتصبح
الاستجابة الهندسية **لفشل المشاريع**،
خسائر اقتصادية جسيمة، وتأخير في الجدول الزمني،
الأسواق التنافسية، وعلى نحو متزايد
متطلب يبحث عن عملاء
الوظائف والجودة والموثوقية في
أقل تكلفة.

Factors of Software Engineering

في مجال البرمجيات، العوامل الأربعة الرئيسية التي تؤثر على هندسة البرمجيات

□ المشروع؛

□ الأشخاص؛ □ العملية؛

□ المنتج .

Project

تحدد عملية إدارة المشروع جميع الأنشطة التي يتعين على إدارة المشروع القيام بها لضمان تحقيق أهداف التكلفة والجودة.

□ التخطيط، □ المراقبة والتحكم،

Project : planning

□ أثناء التخطيط، يتم تحديد الأنشطة الرئيسية مثل تقدير التكلفة، والجدول الزمني وتحديد المعالم، وتوظيف الموظفين في المشروع.

Project : Monitoring and control

□ يتضمن جميع الأنشطة التي يتعين على إدارة المشروع القيام بها أثناء استمرار التطوير لضمان تحقيق أهداف المشروع واستمرار التطوير وفقًا للخطة المطورة

Process

□ مجموعة من الأنشطة التي يكون هدفها
تطوير أو تطوير البرمجيات
□ الأنشطة العامة في جميع عمليات البرمجيات

نكون:

□ المواصفات - ما يجب أن يفعله النظام وقيود تطويره

□ التطوير - إنتاج البرمجيات
نظام

□ التحقق - التأكد من أن البرنامج هو ما يريده العميل

□ التطور - تغيير البرنامج استجابةً للطلبات المتغيرة

People

وظائف/أدوار هندسة البرمجيات

□ **محلل النظم** - يقوم بتحليل المتطلبات الخاصة بـ طلب،

□ **مهندس برمجيات** - يصمم الهيكل العام لـ التطبيق

□ **مبرمج برمجيات** - ينفذ التصميم باستخدام أدوات تطوير البرمجيات،

□ **مسؤول أنظمة البرمجيات** - يدير المستخدم للحديدات التكتيكية والجسور البرمجيات طول مشاكل البرامج

□ **مسؤول قاعدة بيانات البرامج** - يدير قاعدة البيانات (التثبيت والصيانة والنسخ الاحتياطي،
مرطبات)

People

□ **مهندس دعم العملاء** - يحل مشاكل العملاء والمستخدمين النهائيين مع تطبيقات الكمبيوتر والتكوين (على سبيل المثال مزود خدمة الإنترنت) □ **مسؤول الموقع** - يصمم وينفذ ويحافظ على موقع الويب □ **مهندس أمان البرمجيات** - التعريف والتفويض والمصادقة وحماية البيانات وسلامة البيانات □ **مختبر البرمجيات** - التحقق والتحقق المستقل □ **مدير مشروع البرمجيات** - التخطيط والتنظيم والتوجيه والتنسيق والتحكم في مشروع البرمجيات (التركيز على إدارة المخاطر) -

□ **مدير/مهندس جودة البرمجيات** - نمذجة موثوقية البرمجيات، ومراقبة الجودة الإحصائية، وتحليل العيوب

Products

منتج العمل هو البرامج والمستندات والبيانات التي توفرها برامج الكمبيوتر. **تعريف**

المتطلبات: النموذج الأولي السريع، أو وثيقة المتطلبات **المواصفات** وثيقة

المواصفات **التصميم** التصميم المعماري التصميم التفصيلي

Products

□ الترميز

□ الكود المصدر □

حالات الاختبار

□ الاختبار □ اختبار التكامل □

اختبار القبول □ الصيانة

□ تغيير السجل □ حالات اختبار

الانحدار

مشروع برمجي بدون منهجية برمجية هندسة	
نظرة النظرية على أنشطتنا اليومية كالتصميمات.	
المهارة، كالمطورين بالبرمجيات، هي مهارات اللازمة للبرمجة. واحدة أو محدودة .	
تتضمن المنهجيات المثبتة والممارسات الشائعة.	تتضمن المنهجيات المثبتة والممارسات الشائعة.

Software Engineering

مشروع برمجي مدعوم هندسة برمجيات

مهندس البرمجيات هو الشخص الذي يشارك في دورة حياة تطوير البرمجيات، والمصمم، والمبرمج، والمختبر، وحتى المستخدم. إحدى العمليات المدرجة على اليسار.

لا يتم تدريس البرمجيات بالطرق التقليدية، لكنهم يؤمنون بالتعلم من خلال الممارسة. القواعد المشتركة

البرمجة هي تفاعل شطرنج بين جميع الأجزاء والعمليات، بما في ذلك

Software Engineering

مشروع برمجي مع هندسة البرمجيات	مشروع برمجي بدون هندسة البرمجيات
إن المشاكل التي يتعين حلها هي تطوير برمجيات واسعة النطاق لأنظمة معقدة.	المشاكل التي يتعين حلها محدودة على نطاق صغير
يتم تعريف نقل المعرفة وتنفيذه من خلال عمليات هرمية على مستوى المنظمة والمشروع والفرد.	يعد نقل المعرفة في البرمجة أمرًا صعبًا، ويعتبر تصميم وتنفيذ البرامج بمثابة تجربة شخصية .
يمكن إجراء صيانة البرامج بشكل مستقل عن المطورين الأصليين	تعتمد صيانة البرنامج على المصمم الأصلي.

The essential characteristics of Software Engineering (1)

1) هندسة البرمجيات تهتم ببناء التطبيقات الكبيرة. تشير التطبيقات الصغيرة عمومًا إلى التطبيقات التي كتبها شخص واحد في فترة قصيرة نسبيًا.

تشير الطلبات الكبيرة إلى الوظائف التي يشارك فيها عدة أشخاص والتي تمتد لمدة ستة أشهر على الأقل.

الأدوات والتقنيات والطرق اللازمة للتطبيقات الصغيرة تختلف عن تلك اللازمة للتطبيقات الكبيرة

The essential characteristics of Software Engineering (2)

(2الموضوع الرئيسي هو الإتقان
تعقيد

يُضطر المرء إلى تقسيم المشكلة إلى أجزاء بحيث يُمكن استيعاب كل جزء على حدة، مع الحفاظ على بساطة التواصل بين الأجزاء. بهذا الشكل، لا يتناقص التعقيد الكلي، بل يُصبح قابلاً للإدارة.

The essential characteristics of Software Engineering (3)

(3) التعاون المنتظم بين الأشخاص في جزء لا يتجزأ من التطبيقات الكبيرة

نظرًا لأن المشكلة كبيرة، يتعين على العديد من الأشخاص العمل في وقت واحد على حل هذه المشاكل.

يجب أن تكون هناك ترتيبات واضحة ل
توزيع العمل، أساليب العمل

الندم والمسؤوليات وما إلى ذلك.

الترتيبات وحدها لا تكفي، بل يجب على المرء أن
الالتزام بهذه الترتيبات. من أجل فرضها

المعايير أو الإجراءات التي تدعمها

قد يتم استخدام الأدوات.

The essential characteristics of Software Engineering (4)

(٤) البرمجيات تتطور: تُشكّل البرمجيات جزءًا من الواقع. وهذا الواقع يتطور. على سبيل المثال، تتغير لوائح البنوك باستمرار. لذا، يجب أن يتطور البرنامج حتى لا يصبح قديمًا. يجب مراعاة هذه التطورات أثناء التطوير.

The essential characteristics of Software Engineering (5)

(٥) تُعد كفاءة تطوير البرمجيات أمرًا بالغ الأهمية، حيث إن التكلفة الإجمالية ومدة تطوير مشروع البرمجيات مرتفعان. وينطبق هذا أيضًا على صيانة البرمجيات. وتتعلق المواضيع المهمة في مجال هندسة البرمجيات بأساليب وأدوات أفضل وأكثر كفاءة لتطوير وصيانة البرمجيات.

The essential characteristics of Software Engineering (6)

(6) لقد قدم البرنامج دعمًا فعالًا لمستخدميه:

طوّرت البرمجيات لدعم المستخدمين في العمل. يجب أن تتناسب الوظائف المُقدّمة مع مهام المستخدمين. سيحاول المستخدمون غير الراضين عن النظام تجنبه. يعني دعم المستخدم الفعال دراسة المستخدمين في العمل بعناية لتحديد المتطلبات الوظيفية المناسبة، مع مراعاة سهولة الاستخدام وجوانب الجودة الأخرى، مثل الموثوقية والاستجابة وسهولة الاستخدام. وهذا يعني أيضًا أن تطوير البرمجيات لا يقتصر على تقديمها فحسب، بل تُعد أدلة المستخدم ومواد التجهيز مهمة أيضًا.

Software Failure

1968: مؤتمر حول "أزمة البرمجيات".

كان تسليم البرامج يتأخر أحيانًا لسنوات. كانت تكلفتها غالبًا أعلى بكثير من المتوقع. كانت العديد من البرامج غير موثوقة. كانت صيانة البرامج صعبة في الغالب.

غالبًا ما كان أداء البرنامج للمهمة ضعيفًا الذي صمم من أجله.

تم صياغة مصطلح "هندسة البرمجيات".

Software Failure

❑ فشل البرنامج هو مشروع برمجي يحتوي على واحد أو أكثر من الأسباب التالية: • تجاوز الميزانية • التأخر

- لا يلبي احتياجات المستخدم أو توقعاته • لا يلبي المتطلبات الوظيفية أو متطلبات الأداء
- لا يلبي متطلبات الجودة

Software Failure

أزمة البرمجيات:

□ لكل 8 مشاريع كبيرة، يتم إلغاء 2 منها.

□ تتجاوز معظم المشاريع الجدول الزمني الخاص بها بمقدار 50%.

□ 75% من الأنظمة الكبيرة تتعرض للخلل بعد اكتمالها.

□ إن اكتشاف الأخطاء وتصحيحها يستغرق 50% من الوقت والجهد.

□ تكلفة صيانة النظام هي 60% من التكلفة الإجمالية
يكلف.

Software Failure

ما هو سبب فشل البرنامج؟

(1) متطلبات البرمجيات لا تصف بشكل كاف احتياجات المستخدم أو توقعات العملاء. (2) تخطيط المشروع غير واقعي في كثير من الأحيان أو غير مكتمل أو يتم تجاهله. (3) يتم التقليل من تقديرات تكلفة المشروع والجدول الزمني أو يتم إنشاؤها فقط من قبل الإدارة.

Software Failure

4) جودة البرمجيات من الصعب تحديدها وتصميمها وبنائها

5) من الصعب رؤية تقدم تطوير البرمجيات، وغالبًا ما يكون التقدم غير معروف

6) التغييرات في المتطلبات لا تصاحبها تغييرات في
خطط البرمجيات

7) يتم تغيير التصميم دون تغيير المتطلبات

8) لا يتم استخدام المعايير أو توثيقها

Software Failure Example:

برنامج مكوك الفضاء

□ التكلفة: 10 مليار دولار، أي ملايين الدولارات أكثر من المخطط لها

□ الوقت: 3 سنوات متأخرة

الجودة : أُلغي الإطلاق الأول لمركبة كولومبيا بسبب مشكلة في المزامنة مع أجهزة الكمبيوتر الخمسة الموجودة على متن المكوك. عُزي الخطأ إلى تغيير حدث قبل عامين عندما غيّر أحد المبرمجين عامل تأخير في معالج المقاطعة من 50 إلى 80 ميلي ثانية. كان احتمال الخطأ ضئيلاً لدرجة أنه لم يُسبب أي ضرر خلال آلاف الساعات من الاختبار.

Build or Buy

□ إن بناء البرمجيات مكلف للغاية -التكاليف الثابتة مرتفعة للغاية، ويستغرق التطوير الكثير من الوقت والموارد البشرية

□ طالما كانت الاحتياجات متشابهة، فإن توزيع هذه التكاليف على العديد من المؤسسات يجب أن يكون منطقيًا للغاية لكل من البائعين والمشتريين. □ لقد شهدنا زيادة في تنفيذ برامج المؤسسات، وخاصة بين الشركات الكبرى في مجالات التصنيع والتمويل والموارد البشرية.

Build or Buy

□ عند تطوير البرمجيات، أظهرت الأبحاث أن هناك علاقة عكسية بين حجم المشروع واحتمالية إكماله فعليًا

Project Size	People	Time (months)	Success
< \$750k	6	6	55%
\$750k-\$1.5M	12	9	33%
\$1.5M-\$3M	25	12	25%
\$3M-\$6M	40	18	15%
\$6M-\$10M	250+	24+	8%
\$10M+	500+	36+	0%

Figure 4.9.1

Build or Buy

□ ومع ذلك، في العديد من المجالات الأخرى من
التنظيم والتطوير المخصص مستمر.
لماذا يحدث هذا على الرغم من وجود العديد من منتجات
العملاء الجاهزة (COTS) المتاحة؟

Build or Buy

البرمجيات المخصصة - المزايا:

يوفر لك ما تحتاجه بالضبط - لكل شركة قواعدها التجارية الخاصة. غالبًا، لا يمكن حوسبتها إلا من خلال برامج مخصصة.

أنت تملك شفرة المصدر - يمنحك امتلاك شفرة المصدر تحكمًا أكبر في التحسينات المستقبلية. عندما يدرك العملاء قيمة البرامج المخصصة، يصبحون أكثر تطورًا ويجدون بسرعة طرقًا محددة لتحسين سير عمل أعمالهم. عندما تخطر ببالك أفكار ريادية، يمكنك تعديل البرنامج الحالي ليشملها. بالإضافة إلى ذلك ، يُعد امتلاك شفرة المصدر المخصصة أصلًا قيّمًا؛ أصلًا يجب تضمينه في سعر العمل في حال بيعه .

Build or Buy

تقارير أكثر فائدة - لا داعي لاستخدام برنامج إلا إذا كان قادرًا على تحقيق النتائج المرجوة؛ وغالبًا ما يكون ذلك في شكل تقارير. تتيح البرامج المخصصة إنشاء تقارير مفيدة تُستخدم لاتخاذ قرارات عمل ذكية. أنت، صاحب العمل ، تضمن أن التقارير تُقدم لك المعلومات بالطريقة التي تريدها، وليس كما يعتقد البائع أنك تريده.

مركز دعم فني داخلي أكثر كفاءة - سيكون موظفو مركز الدعم الفني أكثر دراية بقواعد العمل المعنية، وسيتمكنون من مساعدة المستخدمين. سيكونون على دراية بالمشكلات الشائعة، والثغرات، والحلول البديلة. هذا أفضل بكثير من الاضطرار إلى شرح مشكلة محددة لموظفي مركز دعم فني جاهز للاستخدام (COTS) الذين يتعاملون عادةً مع مشاكل عامة.

Build or Buy

□ **صناع القرار متاحون بسهولة** - أثناء مرحلة التصميم، يكون صناع القرار متاحين لإصدار الأحكام.

لديهم معرفة دقيقة بكيفية عمل البرنامج. يتميز المستخدمون بمهارة عالية في وصف سير العمل، ونتيجةً لذلك، يُمكن تصميم البرنامج بفعالية أكبر لزيادة كفاءتهم. □ **يتقبل المستخدمون البرنامج بسهولة** - إذا أُعطي المستخدمون آراءهم في تصميم البرنامج الجديد، فسيتقبلون التغييرات بسهولة أكبر. بالإضافة إلى ذلك، فهم متقدمون في منحنى التعلم لأنهم اختبروا البرنامج خلال مرحلة التطوير. □ **لا يُهدر المال على ميزات غير ضرورية** - من فوائد الحصول على الوظيفة التي تريدها بالضبط في البرنامج إنفاق أموال التطوير بحكمة.

كل دولار يتم إنفاقه يذهب إلى صنع أفضل منتج ممكن.

Build or Buy

❑ لا توجد رسوم ترخيص كبيرة - أنت تمتلك الكود.

أنت تملك البرنامج، وتتحكم في استخدامه. هذا لا يعني أن الرسوم السنوية، وتحسينات الأجهزة، وغيرها من نفقات "تكلفة ممارسة الأعمال" غير موجودة. على الأرجح، ستكون موجودة، ولكن عليك معرفة ما تغطيه، ومقدارها، ومقارنتها بنفس رسوم برامج COTS.

Build or Buy

عنوان الموضوعات: قد يكون الخيار الأفضل هو الخيار الأكثر تكلفة - فهو برنامج مخصص أنت وحدك من يقرر ما إذا كانت الفوائد المُستقاة تستحق التكلفة.

□ غير متاح على الفور - اعتمادًا على عدد المطورين ونطاق المشروع وعوامل أخرى، قد يستغرق إنشاء التطبيق شهرًا.

Build or Buy

قد تُعيد اختراع العجلة -فمعظم التطبيقات تحتوي على العديد من الميزات نفسها التي تُوفرها المتطلبات؛ ومن الأمثلة على ذلك طباعة التقارير، وعرض البيانات، وإضافة البيانات أو تعديلها. وستتطلب هذه المهام الأساسية بذل الوقت والمال والجهد .

LAWS ON SOFTWARE ENGINEERING

فيما يلي قائمة بالقوانين المختارة المتعلقة بالبرمجيات الهندسة. □ سيفعل البرنامج دائماً ما تطلب منه القيام به، ولكن

لا يحدث أبداً ما تريد منه أن يفعله.

□ يزداد تعقيد البرنامج حتى يتجاوز

قدرة المبرمج الذي يجب عليه صيانتها.

□ كل برنامج غير تافه لديه خطأ واحد على الأقل.

□ أصغر الأخطاء تسبب أكبر قدر من الضرر

المشاكل.

MURPHY'S LAWS ON SOFTWARE ENGINEERING

□ إضافة العمالة إلى مشروع برمجي متأخر يجعله لاحقاً.

□ البرنامج العامل هو الذي يحتوي على أخطاء غير ملحوظة فقط. □ مهما بلغت الموارد المتاحة لديك، فلن تكون كافية أبداً. □ لن يظهر عطل في الجهاز إلا بعد إصلاحه.

لقد اجتاز التفتيش النهائي.

لا ترتكب أجهزة الكمبيوتر أخطاءً. ما تفعله يفعله عمداً.